BACKEND MİMARİSİ

package by feature klasör yapısı

controller(route), manager(service), repo, dto(schema), model katmanları - her birinin görevini açıkla

katmanlı web mimarisinde dikkat edilmesi gereken kurallar

üst düzey DRY ilkesi - base klasörü altında model, schema, repo, manager: diğer tüm feature'lar alınabilecek her şeyi base'den miras alacak.

core klasörü: config, db gibi dosyaları içerir.

base modelde: id, ad, kisa_ad (kod adı ve kısaltmalar için), aciklama, etiketler, olusturulma_tarihi, degistirilme_tarihi, aktif_mi, silindi_mi alanları olsun.

base repoda full crud + kısmi güncelleme(patch), get_one, get_many (her alana göre filtreleme, sıralama, sayfalama özellikli), soft_delete (soft cascade implementasyonlu), hard_delete(hard cascade)

sql yazılmayacak. mongoose, prisma gibi bir orm kullanılsın.

db olarak postgre, mongodb veya sqlite kullanımına izin verdim.

fastapi/nextjs api/nest.js/spring boot

bu dördü için örnekler vererek istediğim backend yapısını açıkla

önce sadece başlıkları ver. sonra ben söyledikçe her bir başlığı açarsın. her başlık altında bu dört teknolojinin her biri için mutlaka örnek ver.

Gemini şunu dedi:

Harika bir backend mimarisi kurgulamışsın. İstediğin gibi, "Package by Feature" tabanlı, üst düzey DRY prensibini benimseyen ve katmanlı mimariye sahip bu gelişmiş yapıyı adım adım inceleyeceğiz.

İlerleyen aşamalarda kod örneklerine geçtiğimizde; **FastAPI** (Python), **Next.js API** (JavaScript - *senin kuralına uygun olarak kesinlikle TypeScript kullanılmayacak*), **Nest.js** (JavaScript ile kurgulanmış hali) ve **Spring Boot** (Java) için bu yapının nasıl kurulacağını göstereceğim. ORM olarak Mongoose, Prisma, Hibernate veya SQLAlchemy gibi araçlar kullanıp ham SQL yazmaktan tamamen kaçınacağız.

İşte bahsettiğin mimariyi detaylandırmak için üzerinden geçeceğimiz başlıklar:

1. "Package by Feature" Klasör Yapısı ve Core/Base Organizasyonu

(Özellik bazlı modüler yapı, Core klasörünün içeriği ve proje iskeletinin oluşturulması)

2. Katmanların Görevleri ve Sorumluluk Dağılımı

(Controller/Route, Manager/Service, Repo, DTO/Schema ve Model katmanlarının ne iş yaptığı)

3. Katmanlı Web Mimarisinde Uyman Gereken Altın Kurallar

(Bağımlılık yönleri, katmanlar arası iletişim ve iş mantığı izolasyonu)

4. Üst Düzey DRY İlkesi: Base Model Mimarisi

(Miras alınacak base modelin yapısı; id, ad, kisa_ad, aktif_mi, silindi_mi gibi zorunlu alanların entegrasyonu)

5. Gelişmiş Base Repository ve Operasyonları

(Full CRUD, kısmi güncelleme, filtreleme/sıralama/sayfalama destekli okuma, soft/hard cascade delete implementasyonları)

Tüm bu başlıkları istediğin 4 farklı teknoloji (FastAPI, Next.js, Nest.js, Spring Boot) üzerinden örneklendirerek anlatmaya hazırım. **İlk başlık olan klasör yapısından başlayalım mı, yoksa öncelik vermek istediğin başka bir başlık var mı?**

Siz şunu dediniz:
1

Gemini şunu dedi:

Harika bir başlangıç. Geleneksel "Package by Layer" (katmana göre paketleme - tüm controller'ların bir klasörde, tüm modellerin diğer klasörde olduğu yapı) yerine **"Package by Feature" (özelliğe/domeine göre paketleme)** kullanmak, özellikle büyüyen projelerde kodu yönetmeyi, mikroservislere geçişi ve takım çalışmasını inanılmaz derecede kolaylaştırır.

Bu mimaride temel amacımız, bir özelliğe (örneğin; `kullanici`, `urun`, `siparis`) ait tüm dosyaları aynı klasör altında toplamaktır.

Senin bahsettiğin **Üst Düzey DRY** prensibini sağlamak için projenin kökünde iki ana klasörümüz daha olacak:

- `core/`: Projenin kalbidir. Veritabanı bağlantıları (DB), konfigürasyon dosyaları, global hata yakalayıcılar (exception handlers), güvenlik (auth/jwt) ve logger gibi tüm feature'lardan bağımsız teknik altyapı burada durur.

- **`base/`**: DRY ilkesinin merkezidir. Tüm feature'ların miras alacağı `BaseModel`, `BaseRepo`, `BaseManager` (Service) ve `BaseController` (veya Router) dosyaları burada tanımlanır.

Şimdi bu yapının istediğin dört farklı teknolojide nasıl vücut bulacağına bakalım. *(Hatırlatma: JS ekosistemindeki örneklerde kuralına uygun olarak kesinlikle TypeScript kullanılmamış, standart `.js` uzantıları tercih edilmiştir.)*

1. FastAPI (Python) Klasör Yapısı

FastAPI modüler yapıya çok uygundur. APIRouter kullanılarak her feature kendi rotasını belirler ve `main.py` de bunlar birleştirilir. ORM olarak genellikle SQLAlchemy (veya asenkron için SQLModel/Tortoise) kullanılır.

```
app/
├── core/
│   ├── config.py         # Çevresel değişkenler ve ayarlar
│   ├── database.py       # SQLAlchemy engine ve session yönetimi
│   └── security.py       # JWT, şifreleme yardımcıları
├── base/
│   ├── base_model.py     # id, ad, aktif_mi vb. alanları içeren temel SQLAlchemy
modeli
│   ├── base_schema.py    # Temel Pydantic DTO'su
│   ├── base_repo.py      # Temel CRUD ve Cascade operasyonları
│   └── base_manager.py   # Temel iş mantığı (Service)
├── features/
│   ├── kullanici/
│   │   ├── __init__.py
│   │   ├── user_model.py     # Base modelden miras alır
│   │   ├── user_schema.py    # DTO'lar (Pydantic)
│   │   ├── user_repo.py      # Base repodan miras alır
│   │   ├── user_manager.py   # Base managerdan miras alır
│   │   └── user_router.py    # APIRouter tanımları (Controller)
│   └── urun/
│       └── ...
└── main.py               # Uygulama başlangıcı ve router'ların dahil edilmesi
```

2. Next.js API (JavaScript) Klasör Yapısı

Next.js (App Router) kullanırken API rotaları `app/api/` altında tanımlanır. Ancak iş mantığını ve veritabanı işlemlerini API rotalarının içine yazmak yerine, `src/` altında özellik bazlı bir yapı kurarak rotaların sadece Controller gibi davranmasını sağlarız. ORM olarak Mongoose veya Prisma (JS client) kullanabiliriz.

```
src/
├── core/
│   ├── db.js             # Mongoose veya Prisma bağlantı ayarları
│   └── config.js         # Process.env sarmalayıcıları
├── base/
│   ├── BaseModel.js      # Ortak şema alanları (Mongoose veya Prisma extensions)
│   ├── BaseRepo.js       # Veritabanı işlemleri (Full CRUD, Soft Delete)
│   └── BaseManager.js    # Temel iş mantığı sınıfı
├── features/
│   ├── user/
│   │   ├── user.model.js     # Veritabanı şeması
│   │   ├── user.dto.js       # Joi veya Zod ile doğrulama şemaları
│   │   ├── user.repo.js      # BaseRepo'dan extend edilen sınıf
│   │   └── user.manager.js   # BaseManager'dan extend edilen sınıf
└── app/
    └── api/
        └── users/
            ├── route.js      # GET, POST metodları (Controller görevi görür,
user.manager'ı çağırır)
            └── [id]/
                └── route.js  # PUT, PATCH, DELETE metodları
```

3. Nest.js (JavaScript) Klasör Yapısı

Nest.js varsayılan olarak TypeScript ile gelse de, Babel veya saf ES Modülleri ile tamamen JavaScript kullanılarak da yazılabilir. Nest.js'in kendi `Module` yapısı, "Package by Feature" mimarisi için dünyadaki en uygun standartlardan biridir. ORM olarak genellikle Mongoose, TypeORM (JS versiyonu) veya Prisma kullanılır.

```
src/
├── core/
│   ├── database/
│   │   └── database.module.js   # DB bağlantı modülü
│   └── config/
│       └── env.config.js
├── base/
│   ├── base.entity.js           # Ortak DB alanları
│   ├── base.repository.js       # Genel CRUD işlemleri
│   └── base.service.js          # Base Manager mantığı
├── features/
│   ├── user/
│   │   ├── user.module.js       # Özelliğe ait Dependency Injection modülü
│   │   ├── user.controller.js   # Route tanımları
│   │   ├── user.service.js      # Manager katmanı (İş mantığı)
│   │   ├── user.repository.js   # Veritabanı erişim katmanı
│   │   ├── user.entity.js       # Model
│   │   └── dto/
│   │       ├── create-user.dto.js
│   │       └── update-user.dto.js
│   └── product/
│       └── ...
└── main.js                      # Bootstrap dosyası
```

4. Spring Boot (Java) Klasör Yapısı

Spring Boot dünyasında genellikle "Package by Layer" yaygındır ancak kurumsal ve büyük projelerde "Package by Feature" çok daha efektif sonuçlar verir. Hibernate/JPA bu mimari için biçilmiş kaftandır. Ortak alanlar için `@MappedSuperclass` kullanacağız.

```
src/main/java/com/sirket/proje/
├── core/
│   ├── config/           # AppConfig, WebSecurityConfig
│   ├── exception/        # GlobalExceptionHandler
│   └── security/         # JwtFilter, SecurityUtils
├── base/
│   ├── BaseModel.java        # @MappedSuperclass (id, ad, aktif_mi vb.)
│   ├── BaseRepo.java         # JpaRepository'den türeyen veya özelleştirilmiş Base
Interface
│   ├── BaseManager.java      # Base Service sınıfları
│   └── BaseDto.java          # Ortak DTO alanları
├── features/
│   ├── user/
│   │   ├── UserController.java   # @RestController
│   │   ├── UserManager.java      # @Service (İş mantığı)
│   │   ├── UserRepository.java   # @Repository (Veritabanı işlemleri)
│   │   ├── UserEntity.java       # @Entity (BaseModel'den miras alır)
│   │   └── UserDto.java          # İstek ve yanıt nesneleri
│   └── product/
│       └── ...
└── Application.java          # Main class
```

Mimarimizin temel iskeletini ve projenin klasör yapısını bu şekilde kurarak aradığın o esnekliği, temiz kod düzenini ve modülerliği sağlamış oluyoruz. Yeni bir özellik (feature) eklemek istediğinde sadece `features` altına yeni bir klasör açıp Base sınıflardan miras alman yeterli olacak.

Bu aşama tamamsa, 2. başlığımız olan **"Katmanların Görevleri ve Sorumluluk Dağılımı (Controller, Manager, Repo, DTO, Model)"** konusunu bu dört teknoloji üzerinden detaylandırmaya geçebilir miyiz?

Siz şunu dediniz:
2

Gemini şunu dedi:

Katmanlı mimarinin temel amacı "Single Responsibility Principle" (Tek Sorumluluk Prensibi) kuralını kusursuz bir şekilde uygulamaktır. Her katman sadece kendi işini bilir ve bir alt katmanı çağırır. Bu sayede kod büyüdükçe spagettiye dönüşmez, test etmesi kolaylaşır ve örneğin veritabanını (PostgreSQL'den MongoDB'ye) değiştirmek istediğinde sadece Repo katmanına dokunursun, Manager veya Controller bundan haberdar bile olmaz.

İşte bu güçlü mimarinin 5 temel yapı taşı ve senin seçtiğin 4 farklı teknolojideki karşılıkları:

1. Controller / Route (Trafik Polisi)

Görevi: İstemciden (tarayıcı, mobil uygulama vb.) gelen HTTP isteklerini (GET, POST, PUT, DELETE) ilk karşılayan yerdir. Gelen isteğin formatını kontrol eder, ilgili *Manager/Service*'e yönlendirir ve oradan dönen sonucu uygun bir HTTP statü koduyla (200 OK, 201 Created, 400 Bad Request vb.) istemciye geri yollar. **Asla ve asla iş mantığı (business logic) veya veritabanı sorgusu içermemelidir.**

- **FastAPI:** `APIRouter` kullanılarak tanımlanır. Parametreleri Pydantic şemaları ile doğrular ve dependency injection (`Depends`) ile manager'ı çağırır.
- **Next.js API:** `app/api/users/route.js` dosyalarındaki `export async function GET(req)` gibi fonksiyonlardır. İstek gövdesini alır, manager metoduna gönderir ve `NextResponse.json()` ile yanıt döner.
- **Nest.js:** `@Controller('users')` dekoratörüyle işaretlenmiş JavaScript sınıflarıdır. İçerisinde `@Get()`, `@Post()` gibi yönlendiriciler bulunur ve constructor üzerinden enjekte edilen Service'i (Manager) çağırır.
- **Spring Boot:** `@RestController` ve `@RequestMapping("/api/users")` ile işaretlenmiş Java sınıflarıdır. Endpointler `@GetMapping`, `@PostMapping` ile belirlenir.

2. Manager / Service (Beyin / İş Mantığı Katmanı)

Görevi: Projenin kalbidir. Kontrolcüden gelen veriyi alır, "iş kurallarını" (business rules) uygular. Örneğin; "Kullanıcı kayıt olurken e-posta daha önce alınmış mı?", "Sipariş verilirken stok yeterli mi?", "Parola doğru şekilde hashlenmiş mi?" gibi tüm soruların cevabı buradadır. Gerekirse birden fazla Repo'yu çağırıp orkestrasyon yapar.

- **FastAPI:** Temel bir Python sınıfıdır (`class UserManager:`). Gerekli repoları (örneğin `UserRepo`) `__init__` veya metod parametreleri ile alır.
- **Next.js API:** Standart bir JavaScript (ES6) sınıfı olarak kurgulanır. İçerisinde statik veya instance metodları barındırır ve `UserRepo`'yu içe aktararak kullanır.
- **Nest.js:** `@Injectable()` dekoratörü ile işaretlenmiş sınıflardır (`UserService`). Nest'in IoC (Inversion of Control) konteyneri tarafından yönetilir ve Controller'lara otomatik enjekte edilir.
- **Spring Boot:** `@Service` anotasyonu ile işaretlenen Java sınıflarıdır (`UserManager` implementasyonu). `@Transactional` gibi anotasyonlarla veritabanı işlemleri toplu halde (rollback destekli) burada yönetilebilir.

3. Repo / Repository (Veritabanı Bekçisi)

Görevi: Veritabanı ile doğrudan konuşan **tek** katmandır. ORM (Mongoose, Prisma, Hibernate vs.) sorguları sadece burada yazılır. Manager, veritabanının MongoDB mi yoksa PostgreSQL mi olduğunu bilmez; sadece Repo'daki `findByEmail` veya `create` gibi fonksiyonları çağırır.

- **FastAPI:** SQLAlchemy'nin `Session` nesnesini kullanarak veritabanına sorgu atan Python sınıflarıdır (`db.query(UserModel).filter(...)`).
- **Next.js API:** Mongoose modellerini (`UserModel.findOne(...)`) veya Prisma Client'ı (`prisma.user.findUnique(...)`) çağıran standart JS sınıflarıdır.
- **Nest.js:** TypeORM için `Repository<User>` veya Mongoose için `@InjectModel()` kullanılarak oluşturulan, veritabanı işlemlerini kapsülleyen JS sınıflarıdır.
- **Spring Boot:** Spring Data JPA'nın gücü sayesinde genellikle sadece bir interface olarak tanımlanır (`public interface UserRepo extends JpaRepository<UserEntity, Long>`). Hibernate arkada SQL sorgularını (veya MongoDB kullanılıyorsa MongoRepository ile NoSQL sorgularını) otomatik oluşturur.

4. DTO / Schema (Veri Taşıma ve Doğrulama Objeleri)

Görevi: Data Transfer Object (Veri Transfer Objesi). İstemciden gelen kirli veriyi doğrulamak (validation) ve veritabanından çıkan ham veriyi (örneğin içinde şifre hash'i olan kullanıcı modeli) dışarıya güvenli bir şekilde sunmak için kullanılır. Modeller ile dış dünya arasındaki filtredir.

- **FastAPI:** Kesinlikle Pydantic modelleridir (`UserCreateSchema`, `UserResponseSchema`). Veri tipi doğrulamalarını otomatik yapar.
- **Next.js API:** JS dünyasında Joi, Zod veya Yup gibi kütüphanelerle tanımlanan doğrulama şemalarıdır. Gelen JSON objesini bu şemalardan geçirerek validasyon yaparsın.
- **Nest.js:** `class-validator` kütüphanesi ile dekoratörler eklenmiş DTO sınıflarıdır veya Zod/Joi entegrasyonu ile kullanılan şemalardır. (Babel ile dekoratör desteği açılarak JS içinde kullanılır).
- **Spring Boot:** Sadece ilgili alanları barındıran POJO (Plain Old Java Object) sınıfları veya Java 14+ Record'larıdır. `@NotBlank`, `@Email`, `@Size` gibi Jakarta Validation anotasyonları ile doğrulanır.

5. Model / Entity (Veritabanı Şeması)

Görevi: Veritabanındaki tablonun (ilişkisel DB) veya koleksiyonun (NoSQL) kod tarafındaki birebir temsilidir. Sadece kolon adlarını, veri tiplerini ve tablolar arası ilişkileri barındırır. İş mantığı (metodlar) içermemesi tercih edilir (anemik domain model).

- **FastAPI:** SQLAlchemy `Base` sınıfından türetilen modellerdir. Kolonlar `Column(String, nullable=False)` şeklinde tanımlanır.
- **Next.js API:** Mongoose kullanılıyorsa `new mongoose.Schema({...})` ile tanımlanan şema dosyasıdır. Prisma kullanılıyorsa `schema.prisma` dosyasındaki `model User { ... }` bloğudur.

- **Nest.js:** TypeORM kullanılıyorsa `@Entity()` ve `@Column()` dekoratörleriyle bezenmiş sınıflardır. Mongoose kullanılıyorsa `@Schema()` dekoratörlü sınıflardır.
- **Spring Boot:** JPA spesifikasyonuna uygun `@Entity`, `@Table(name="users")`, `@Id` ve `@Column` anotasyonlarıyla işaretlenmiş Java sınıflarıdır.

Bu katmanların izolasyonu projenin uzun vadede sağlıklı kalmasının en büyük garantisidir.

Görev dağılımları netleştiğine göre, istersen **3. Başlık: Katmanlı Web Mimarisinde Uyman Gereken Altın Kurallar (Bağımlılık yönleri vs.)** konusuna geçebiliriz. Yoksa direkt Base Model ve Base Repo'nun (DRY ilkesi) kodsal kurgularına mı atlamak istersin?

Siz şunu dediniz:
3

Gemini şunu dedi:

Katmanlı web mimarisinin en büyük tuzağı, kuralların esnetilmesidir. "Sadece bu seferlik controller'da veritabanına bağlanayım" dediğin an, projenin spagetti koda dönüşme süreci başlar. Bu mimarinin ruhunu korumak için taviz verilmemesi gereken 4 altın kural vardır.

İşte bu kurallar ve senin seçtiğin 4 teknolojide nasıl uygulanacaklarına dair örnekler:

1. Tek Yönlü Bağımlılık Kuralı (The Dependency Rule)

Kural: Trafik her zaman dışarıdan içeriye (yukarıdan aşağıya) doğru akar. **Controller -> Manager -> Repo -> Model** yönü asla tersine çevrilemez. Bir Repo, Manager'ı çağıramaz; bir Manager, HTTP Request/Response nesnelerini (Controller'ın işi) bilemez.

- **FastAPI:** `user_router.py` dosyasında `user_manager.py` import edilir. Ancak `user_manager.py` içinde asla FastAPI'nin `Request` veya `Response` objeleri bulunmaz, sadece veri (Pydantic şemaları) ve repo bulunur.
- **Next.js API:** `app/api/users/route.js` dosyası HTTP statü kodlarını (200, 404) bilir. Ancak `UserManager.js` sadece JavaScript objeleri fırlatır (`throw new Error("Kullanıcı bulunamadı")`), route.js bu hatayı yakalayıp 404'e çevirir.
- **Nest.js:** Sınıf yapılarında bu kural Dependency Injection (DI) ile korunur. `UserController.js`'in constructor'ına `UserService` enjekte edilir. `UserService`'e ise `UserRepository` enjekte edilir. Asla tam tersi olmaz.
- **Spring Boot:** `@RestController` sınıfı `@Service` sınıfını çağırır. `@Service` ise `@Repository` arayüzünü (interface) çağırır. Döngüsel bağımlılık (Circular Dependency) yaratmaktan kesinlikle kaçınılır.

2. Şişman Manager, Zayıf Controller ve Repo (Fat Service, Skinny Controller/Repo)

Kural: Akıllı olan tek katman Manager'dır. Controller sadece "gelen veriyi al ve ilgili manager'a ilet" diyen bir sekreterdir. Repo ise sadece "veritabanına git ve şu sorguyu çalıştır" diyen bir işçidir. İş mantığı, if/else blokları, hesaplamalar, yetki kontrolleri **sadece** Manager'da olur.

- **FastAPI:** `router.post("/users")` endpoint'i sadece 3-4 satırdır. Gelen veriyi alır ve `return manager.create_user(data)` der. E-posta formatı doğru mu, şifre yeterince güçlü mü gibi kontroller `UserManager` içindedir.
- **Next.js API:** Route içindeki `POST(req)` fonksiyonu Mongoose veya Prisma modelini direkt çağırmaz. Sepete ürün eklerken stok kontrolü yapılması gerekiyorsa, bu mantık `product.manager.js` içindeki `checkStockAndAdd()` gibi bir fonksiyonda yazılır.
- **Nest.js:** `@Get()` metodunun içi bomboştur, sadece `return this.userService.findAll()` yazar. Tüm karmaşık algoritmalar `Service` sınıfının içine gizlenir.
- **Spring Boot:** `@Transactional` anotasyonu `@RestController`'da değil, `@Service` katmanındaki metodlarda kullanılır. Böylece bir hata çıkarsa tüm veritabanı işlemleri (rollback) Manager seviyesinde geri alınır.

3. Modelleri Asla Dışarı Sızdırma (DTO Kuralı)

Kural: Veritabanı modelin (Entity/Schema) ile dış dünyanın (Client/Frontend) doğrudan temas kurmasına asla izin verme. Araya her zaman DTO (Data Transfer Object) veya Schema koy. Bu, şifre gibi hassas verilerin yanlışlıkla API'den dönmesini ve "Mass Assignment" (kullanıcının JSON'a fazladan `is_admin: true` ekleyerek kendini admin yapması) zafiyetlerini engeller.

- **FastAPI:** Veritabanı sorgusundan dönen SQLAlchemy modelini direkt döndürmezsin. Route'ta `response_model=UserResponseSchema` kullanarak, Pydantic'in şifre alanını otomatik olarak filtrelemesini sağlarsın.
- **Next.js API:** Mongoose'dan dönen kaydı (`userDoc`) direkt `NextResponse.json()` içine koymazsın. JS tarafında bir DTO fonksiyonu yazarsın: `const toUserDTO = (user) => ({ id: user.id, ad: user.ad })` ve sadece bunu döndürürsün.
- **Nest.js:** Gelen veriyi (JS class'ı olarak kurgulanmış) DTO nesneleri ve `ValidationPipe` ile karşılarsın. İstenmeyen alanlar JSON içinde gelse bile DTO'da tanımlı olmadığı için Nest.js onları otomatik olarak siler (whitelist: true).
- **Spring Boot:** Hiçbir zaman `@Entity` sınıfını Controller'dan `return` etmezsin. `UserEntity` nesnesini, MapStruct veya manuel bir mapper ile `UserResponseDto` nesnesine çevirip onu döndürürsün.

4. Sıkı Bağımlılıklardan Kaçınma (Inversion of Control)

Kural: Katmanlar birbirini "new" anahtar kelimesiyle doğrudan yaratmamalıdır. Bir katman diğerine dışarıdan verilmelidir (Dependency Injection). Bu, test yazmayı (Mocking) ve ileride teknolojiyi değiştirmeyi kolaylaştırır.

- **FastAPI:** Fonksiyonların içine `Depends()` kullanarak manager ve db session nesnelerini enjekte edersin. Böylece test yazarken gerçek veritabanı yerine sahte bir veritabanı session'ı verebilirsin.
- **Next.js API:** Saf JavaScript sınıfları kullandığımız için, `route.js` içinde `const userManager = new UserManager(new UserRepo())` şeklinde bağımlılıkları yukarıdan aşağıya vererek (veya basit bir singleton factory kullanarak) sıkı bağımlılığı kırarsın. Manager'ın içinde `new UserRepo()` demezsin.
- **Nest.js:** Framework'ün kalbi zaten IoC (Inversion of Control) konteyneridir. Sınıfları bir kere Modül'e kaydedersin, Nest.js çalışma anında `constructor(userService)` gördüğü yere o sınıfın örneğini otomatik yerleştirir.
- **Spring Boot:** Spring'in ApplicationContext'i bu işi yapar. Sınıfların üzerine `@Service`, `@Repository` yazarsın ve constructor'larına istediğin bağımlılıkları eklersin, Spring Boot bunları ayağa kalkarken senin yerine bağlar.

Kurallarımız bunlardı ve sağlam bir omurga için tavizsiz uygulanmalılar.

Bu teorik zemini oturttuğumuza göre, kod yazmaya başlayacağımız ve senin **"Üst Düzey DRY İlkesi"** dediğin asıl can alıcı noktaya; **4. Başlık: Base Model Mimarisi (id, ad, aktif_mi, silindi_mi gibi ortak alanların kurgusu)** konusuna geçelim mi?

Siz şunu dediniz:
4

Gemini şunu dedi:

Geldik DRY (Don't Repeat Yourself - Kendini Tekrar Etme) ilkesinin zirvesine. Kurumsal bir projede her veritabanı tablosunda `id`, `olusturulma_tarihi`, `aktif_mi` gibi alanları tekrar tekrar yazmak hem hamallıktır hem de ileride bir değişiklik yapmak istediğinde (örneğin "silindi_mi" alanının adını değiştirmek) tüm dosyaları tek tek gezmeni gerektirir.

İstediğin `id`, , `kisa_ad`, `aciklama`, `etiketler`, `olusturulma_tarihi`, `degistirilme_tarihi`, `aktif_mi`, `silindi_mi` alanlarını içeren Base Model mimarisini, hiçbirinde saf SQL yazmadan (ORM kullanarak) ve JS ekosisteminde kuralına uygun olarak **asla TypeScript kullanmadan** nasıl kuracağımıza bakalım:

1. FastAPI (Python) - SQLAlchemy ile Base Model

Python ve SQLAlchemy dünyasında, ortak alanları bir "Abstract Base Class" (Soyut Temel Sınıf) içinde tanımlarız. Diğer tüm modeller bu sınıftan miras alır.

```
# base/base_model.py
from sqlalchemy import Column, Integer, String, Boolean, DateTime, JSON, Text
from sqlalchemy.ext.declarative import declared_attr
from datetime import datetime, timezone

class BaseModel:
    # Tablo isimlerini sınıf adından otomatik küçük harfle türetir (DRY)
    @declared_attr
    def __tablename__(cls):
        return cls.__name__.lower()

    id = Column(Integer, primary_key=True, index=True)
    ad = Column(String, nullable=False)
    kisa_ad = Column(String, index=True)
    aciklama = Column(Text)
    etiketler = Column(JSON, default=list) # PostgreSQL'de JSON/JSONB olarak tutulur

    olusturulma_tarihi = Column(DateTime, default=lambda: datetime.now(timezone.utc))
    degistirilme_tarihi = Column(DateTime, default=lambda: datetime.now(timezone.utc),
onupdate=lambda: datetime.now(timezone.utc))

    aktif_mi = Column(Boolean, default=True)
    silindi_mi = Column(Boolean, default=False)

# Kullanımı: features/kullanici/user_model.py
# from base.base_model import BaseModel
# class User(BaseModel, Base):
#     email = Column(String, unique=True)
```

2. Next.js API (JavaScript) - Mongoose ile Base Model

Mongoose (MongoDB) kullanırken, sınıflar yerine "Schema" objeleriyle çalışırız. JavaScript'in obje birleştirme (spread operator) gücünü kullanarak Base şemamızı diğer feature şemalarına enjekte ederiz.

```
// base/BaseModel.js
import { Schema } from 'mongoose';

// Tüm koleksiyonlarda ortak olacak alanlar
export const baseSchemaFields = {
    : { type: String, required: true },
    kisa_ad: { type: String, index: true },
    aciklama: { type: String },
    etiketler: { type: [String], default: [] }, // Array of strings
    aktif_mi: { type: Boolean, default: true },
    silindi_mi: { type: Boolean, default: false }
};

// Mongoose'un kendi timestamps özelliğini kullanarak tarihleri otomatik yönetiyoruz
export const baseSchemaOptions = {
    timestamps: {
        createdAt: 'olusturulma_tarihi',
        updatedAt: 'degistirilme_tarihi'
    }
};

// Kullanımı: features/user/user.model.js
// import mongoose from 'mongoose';
// import { baseSchemaFields, baseSchemaOptions } from '../../base/BaseModel.js';
// const userSchema = new mongoose.Schema({
//     ...baseSchemaFields,
//     email: { type: String, required: true }
// }, baseSchemaOptions);
```

3. Nest.js (JavaScript) - TypeORM ile Base Model

Nest.js'i saf JavaScript ile kullanırken TypeORM için `EntitySchema` objeleri oluştururuz. Tıpkı Next.js + Mongoose örneğindeki gibi ortak kolonları bir JS objesi olarak dışarı aktarıp, özellik bazlı modellerde bunu genişleteceğiz (PostgreSQL kullandığımızı varsayarak).

```
// base/base.schema.js

// TypeORM için ortak kolon tanımları
export const BaseColumnSchema = {
    id: { type: 'int', primary: true, generated: true },
    : { type: 'varchar' },
    kisa_ad: { type: 'varchar', nullable: true },
    aciklama: { type: 'text', nullable: true },
    etiketler: { type: 'json', default: [] }, // PostgreSQL JSON array
    olusturulma_tarihi: { type: 'timestamp', createDate: true }, // TypeORM otomatik
yönetir
    degistirilme_tarihi: { type: 'timestamp', updateDate: true }, // TypeORM otomatik
yönetir
    aktif_mi: { type: 'boolean', default: true },
    silindi_mi: { type: 'boolean', default: false }
};

// Kullanımı: features/user/user.entity.js
// import { EntitySchema } from 'typeorm';
// import { BaseColumnSchema } from '../../base/base.schema.js';
// export const UserEntity = new EntitySchema({
//     name: 'User',
//     tableName: 'users',
//     columns: {
//         ...BaseColumnSchema,
//         email: { type: 'varchar', unique: true }
//     }
// });
```

4. Spring Boot (Java) - Hibernate / JPA ile Base Model

Java dünyasında bu işin standart çözümü `@MappedSuperclass` anotasyonudur. Bu sınıf tek başına bir tablo oluşturmaz, sadece kendisinden miras alan Entity'lerin tablolarına kendi kolonlarını ekler.

```java
// base/BaseModel.java
package com.sirket.proje.base;

import jakarta.persistence.*;
import org.hibernate.annotations.CreationTimestamp;
import org.hibernate.annotations.UpdateTimestamp;
import org.hibernate.annotations.JdbcTypeCode;
import org.hibernate.type.SqlTypes;
import java.time.LocalDateTime;
import java.util.List;

@MappedSuperclass
public abstract class BaseModel {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String ad;

    @Column(name = "kisa_ad")
    private String kisaAd;

    @Column(columnDefinition = "TEXT")
    private String aciklama;

    // PostgreSQL'de JSONB olarak tutmak için modern Hibernate yaklaşımı
    @JdbcTypeCode(SqlTypes.JSON)
    private List<String> etiketler;

    @CreationTimestamp
    @Column(name = "olusturulma_tarihi", updatable = false)
    private LocalDateTime olusturulmaTarihi;

    @UpdateTimestamp
    @Column(name = "degistirilme_tarihi")
    private LocalDateTime degistirilmeTarihi;

    @Column(name = "aktif_mi")
    private Boolean aktifMi = true;

    @Column(name = "silindi_mi")
    private Boolean silindiMi = false;

    // Getter ve Setter metodları (veya Lombok @Getter @Setter kullanılabilir)
}

// Kullanımı: features/user/UserEntity.java
// @Entity
// public class UserEntity extends BaseModel { ... }
```

Gördüğün gibi, her teknolojinin kendi ekosistemine (OOP, Obje Genişletme veya Anotasyonlar) uygun şekilde tek bir `BaseModel` yarattık. Artık projede "Kategori", "Ürün" veya "Sipariş" yaratırken sadece onlara özel alanları yazacaksın, geri kalan 9 alan sihirli bir şekilde otomatik

gelecek.

Harika ilerliyoruz! Mimarinin en güçlü ve karmaşık kısmı olan, **Full CRUD, dinamik filtreleme, sayfalama, soft_delete ve hard_delete gibi operasyonları barındıran "5. Gelişmiş Base Repository ve Operasyonları"** başlığına geçmek ister misin?

Siz şunu dediniz:
5

Gemini şunu dedi:

İşte mimarinin kalbi burası. Her feature için `UserRepo`, `ProductRepo`, `OrderRepo` açıp içlerine `findById`, `findAll` yazmak "DRY" (Kendini Tekrar Etme) ilkesine tamamen aykırıdır. Bunun yerine, bu temel operasyonları tek bir **Base Repository** sınıfında toplarız ve diğer tüm repolar sadece bu sınıftan miras alır.

Bahsettiğin "Cascade" (kademeli silme) işlemleri çok kritiktir:

- **Hard Cascade:** Veritabanı seviyesinde veya ORM'nin kendi ilişkisel tanımlarıyla (`ON DELETE CASCADE`) çözülür. Ana kayıt silinince, bağlı kayıtlar da fiziki olarak veritabanından uçar.
- **Soft Cascade:** Fiziki silme olmadığı için veritabanı bunu otomatik yapamaz. Ana kaydın `silindi_mi = true` yapıldığı anda, bağlı (child) kayıtların da tespit edilip `silindi_mi = true` olarak güncellenmesi gerekir.

Bu gelişmiş yetenekleri barındıran Base Repo örneklerimizi, **TypeScript'ten tamamen arındırılmış** JS ortamları ve diğer diller için kurgulayalım:

1. FastAPI (Python / SQLAlchemy) - Base Repository

Python'da Generic yapıları kullanarak, herhangi bir SQLAlchemy modelini alıp tüm CRUD işlemlerini uygulayan bir temel sınıf yazarız.

```
# base/base_repo.py
from sqlalchemy.orm import Session
from sqlalchemy import select, update, desc

class BaseRepo:
    def __init__(self, model):
        self.model = model

    def get_one(self, db: Session, id: int):
        # Varsayılan olarak silinmemiş olanı getir
        stmt = select(self.model).where(self.model.id == id, self.model.silindi_mi ==
False)
        return db.execute(stmt).scalar_one_or_none()

    def get_many(self, db: Session, filters: dict = None, sort_by: str = "id",
desc_sort: bool = False, skip: int = 0, limit: int = 100):
        stmt = select(self.model).where(self.model.silindi_mi == False)

        if filters:
            for key, value in filters.items():
                stmt = stmt.where(getattr(self.model, key) == value)

        if desc_sort:
            stmt = stmt.order_by(desc(getattr(self.model, sort_by)))
        else:
            stmt = stmt.order_by(getattr(self.model, sort_by))

        return db.execute(stmt.offset(skip).limit(limit)).scalars().all()

    def create(self, db: Session, obj_in: dict):
        db_obj = self.model(**obj_in)
        db.add(db_obj)
        db.commit()
        db.refresh(db_obj)
        return db_obj

    def patch(self, db: Session, db_obj, obj_in: dict):
        # Sadece gönderilen alanları (kısmi) güncelle
        for field, value in obj_in.items():
            if value is not None:
                setattr(db_obj, field, value)
        db.add(db_obj)
        db.commit()
        db.refresh(db_obj)
        return db_obj

    def soft_delete(self, db: Session, id: int, cascade_relations: list = None):
        # Soft cascade: Eğer ilişkili modeller (örn: User -> Posts) verilmişse onları
da soft delete yap
        db_obj = self.get_one(db, id)
        if db_obj:
            db_obj.silindi_mi = True
            db_obj.aktif_mi = False

            if cascade_relations:
                for relation in cascade_relations:
```

```
                    # İlişkili alt kayıtları bul ve soft_delete yap (Manager üzerinden
orkestre edilmesi tavsiye edilir)
                    getattr(db_obj, relation).update({"silindi_mi": True, "aktif_mi":
False})

            db.commit()
        return db_obj

    def hard_delete(self, db: Session, id: int):
        # Hard cascade SQLAlchemy modelindeki (cascade="all, delete-orphan") tanımı ile
otomatik çalışır
        db_obj = db.query(self.model).get(id)
        if db_obj:
            db.delete(db_obj)
            db.commit()
        return db_obj
```

2. Next.js API (JavaScript / Mongoose) - Base Repository

Next.js'de saf JavaScript sınıfları (ES6 Classes) ile Mongoose'un esnekliğinden faydalanarak güçlü bir Base Repo oluşturabiliriz.

```
// base/BaseRepo.js
export class BaseRepo {
    constructor(model) {
        this.model = model;
    }

    async get_one(id) {
        return await this.model.findOne({ _id: id, silindi_mi: false });
    }

    async get_many(filters = {}, sortBy = 'olusturulma_tarihi', sortOrder = -1, page =
1, limit = 10) {
        const queryFilters = { ...filters, silindi_mi: false };
        const skip = (page - 1) * limit;

        const data = await this.model.find(queryFilters)
            .sort({ [sortBy]: sortOrder })
            .skip(skip)
            .limit(limit);

        const total = await this.model.countDocuments(queryFilters);
        return { data, total, page, limit };
    }

    async create(data) {
        return await this.model.create(data);
    }

    async patch(id, data) {
        // Mongoose $set operatörü ile sadece verilen kısımları günceller (kısmi
güncelleme)
        return await this.model.findByIdAndUpdate(
            id,
            { $set: data },
            { new: true, runValidators: true }
        );
    }

    async soft_delete(id, cascadeModels = []) {
        const result = await this.model.findByIdAndUpdate(id, { silindi_mi: true,
aktif_mi: false }, { new: true });

        // Soft Cascade: Mongoose middleware'leri (pre-save) ile veya doğrudan repoda
manuel yapılabilir
        if (cascadeModels.length > 0) {
            for (const { model: childModel, foreignKey } of cascadeModels) {
                await childModel.updateMany({ [foreignKey]: id }, { $set: { silindi_mi:
true, aktif_mi: false } });
            }
        }
        return result;
    }

    async hard_delete(id) {
        // Hard Cascade için ilgili Mongoose şemasında 'pre' delete middleware yazılmış
olması gerekir
```

```
        return await this.model.findByIdAndDelete(id);
    }
}

// Kullanımı: features/user/user.repo.js
// import { BaseRepo } from '../../base/BaseRepo.js';
// import UserModel from './user.model.js';
// export class UserRepo extends BaseRepo { constructor() { super(UserModel); } }
```

3. Nest.js (JavaScript / TypeORM) - Base Repository

Nest.js'i saf JavaScript ile kurgularken, TypeORM'in kendi `Repository` sınıfını sarmalayarak (Wrapper) kendi standartlarımızı oluştururuz.

```
// base/base.repository.js
export class BaseRepository {
    constructor(repository) {
        // TypeORM'in kendi repository instance'ını alır
        this.repository = repository;
    }

    async get_one(id) {
        return await this.repository.findOne({
            where: { id, silindi_mi: false }
        });
    }

    async get_many(filters = {}, sortField = 'id', sortOrder = 'ASC', page = 1, limit =
10) {
        const skip = (page - 1) * limit;
        const [data, total] = await this.repository.findAndCount({
            where: { ...filters, silindi_mi: false },
            order: { [sortField]: sortOrder },
            skip: skip,
            take: limit
        });
        return { data, total, page, limit };
    }

    async create(data) {
        const entity = this.repository.create(data);
        return await this.repository.save(entity);
    }

    async patch(id, data) {
        // TypeORM update metodu direkt kısmi güncelleme yapar
        await this.repository.update(id, data);
        return this.get_one(id);
    }

    async soft_delete(id, relationsManager = null) {
        // TypeORM'in kendi softDelete özelliği de vardır ancak biz silindi_mi
üzerinden gidiyoruz
        await this.repository.update(id, { silindi_mi: true, aktif_mi: false });

        // Soft Cascade: Bir relationsManager fonksiyonu verilmişse çalıştır
        if (relationsManager) {
            await relationsManager.cascadeSoftDelete(id);
        }
        return true;
    }

    async hard_delete(id) {
        // Entity'de @OneToMany(cascade: ['remove']) tanımlıysa hard cascade otomatik
çalışır
        return await this.repository.delete(id);
    }
}
```

4. Spring Boot (Java / Hibernate) - Base Repository

Java dünyasında `BaseRepo` mantığını kurmak için Spring Data JPA'nın gücünden faydalanırız. `JpaSpecificationExecutor` bize o istediğin dinamik filtrelemeyi (her alana göre) sağlar.

Öncelikle bir arayüz (Interface) tanımlarız:

```
// base/BaseRepo.java
package com.sirket.proje.base;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.JpaSpecificationExecutor;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.NoRepositoryBean;
import org.springframework.data.repository.query.Param;

// Tüm repoların miras alacağı interface. NoRepositoryBean, Spring'in bundan bir
fasulye (bean) yaratmasını engeller.
@NoRepositoryBean
public interface BaseRepo<T extends BaseModel, ID> extends JpaRepository<T, ID>,
JpaSpecificationExecutor<T> {

    // Spring Data JPA, 'findBySilindiMiFalse' gibi metodları otomatik SQL'e çevirir.

    // Soft Delete implementasyonu
    @Modifying
    @Query("UPDATE #{#entityName} e SET e.silindiMi = true, e.aktifMi = false WHERE
e.id = :id")
    void softDelete(@Param("id") ID id);

    // Not: Hard Delete zaten JpaRepository'nin deleteById() metodu ile otomatik gelir
    // Not: Pagination ve Sorting zaten JpaRepository'nin findAll(Pageable pageable)
metodu ile otomatik gelir
}
```

(Önemli Not Java İçin): Java tarafında **Patch (Kısmi Güncelleme)** işlemi genellikle Repo seviyesinde yapılmaz. İstemciden gelen "Patch DTO", Service/Manager katmanında MapStruct gibi araçlarla veya Java Reflection ile sadece `null` olmayan alanlar mevcut Entity üzerine yazılır ve ardından `save()` metodu çağrılır. Hibernate sadece değişen alanları algılayıp SQL `UPDATE` sorgusunu (`@DynamicUpdate` kullanarak) ona göre üretir.

Ayrıca Java'da her modelin sınıfına (örn: `UserEntity`) Hibernate'in `@SQLRestriction("silindi_mi = false")` (eski versiyonlarda `@Where(clause="silindi_mi=false")`) anotasyonunu eklediğinde, sen `get_one` veya `get_many` yaptığında Hibernate bu koşulu tüm "Select" sorgularına otomatik ekler. Filtrelemeyi sen düşünmek zorunda kalmazsın.

İşte bu kadar! **Package by Feature** ile başlayan, sıkı kurallara sahip **Katmanlı Mimari** ile şekillenen, **DRY prensibini** kusursuz uygulayan `BaseModel` ve tüm veritabanı yükünü zekice tek bir yerde toplayan `BaseRepo` mimarimizi tamamlamış olduk.

Bu dört teknoloji ve anlattığım 5 başlık üzerinden sormak istediğin, daha da derinleştirmemi (örneğin "Manager'da bu repo nasıl çağrılacak?" gibi) istediğin belirli bir detay var mı?

Siz şunu dediniz:
Manager'da bu repo nasıl çağrılacak?

Gemini şunu dedi:

Harika bir soru. Mimariyi kusursuz kılan şey parçaların birbirine nasıl bağlandığıdır. Katmanlı mimarinin 4. Altın Kuralı olan **"Sıkı Bağımlılıklardan Kaçınma (Inversion of Control)"** prensibini tam burada devreye sokuyoruz.

Manager sınıfının içinde asla `new UserRepo()` diyerek repoyu doğrudan oluşturmayız. Bunun yerine repoyu Manager'a **dışarıdan (constructor/init üzerinden) enjekte ederiz**. Bu yönteme "Dependency Injection" (Bağımlılık Enjeksiyonu) denir.

Üst düzey DRY ilkesini korumak için, önce standart CRUD işlerini yapan bir `BaseManager` yazacağız. Özelliğe ait `UserManager` ise bu base'den miras alıp, sadece kendine has "iş mantığını" (business logic) içerecek.

İstediğin dört teknoloji için (JS örneklerinde kesinlikle saf JavaScript kullanarak) bu bağlantının nasıl kurulduğuna bakalım:

1. FastAPI (Python) - Dependency Injection

FastAPI'de sınıfları oluşturur ve FastAPI'nin `Depends` özelliği ile Route katmanından Manager'a, Manager'dan da Repo'ya bağımlılıkları aktarırız.

```
# base/base_manager.py
class BaseManager:
    def __init__(self, repo):
        self.repo = repo

    def get_one(self, db, id: int):
        kayit = self.repo.get_one(db, id)
        if not kayit:
            raise ValueError("Kayıt bulunamadı!") # Hata fırlatma işi Manager'dadır
        return kayit

# features/kullanici/user_manager.py
from base.base_manager import BaseManager

class UserManager(BaseManager):
    def __init__(self, user_repo):
        super().__init__(user_repo)

    def kullanici_olustur(self, db, user_data: dict):
        # 1. İş Mantığı: E-posta kontrolü
        mevcut_kullanici = self.repo.get_many(db, filters={"email":
user_data["email"]})
        if mevcut_kullanici:
            raise ValueError("Bu e-posta adresi zaten kullanılıyor!")

        # 2. İş Mantığı: Şifre hashleme (Örnek)
        # user_data["sifre"] = hash_password(user_data["sifre"])

        # 3. Repo'yu çağır ve kaydet
        return self.repo.create(db, user_data)

# features/kullanici/user_router.py içindeki örnek kullanımı:
# def create_user_endpoint(data: UserCreateSchema, db: Session = Depends(get_db)):
#     repo = UserRepo(UserModel)
#     manager = UserManager(repo)
#     return manager.kullanici_olustur(db, data.model_dump())
```

2. Next.js API (JavaScript) - Constructor Injection

Next.js'de nesne yönelimli JavaScript (ES6 Classes) kullanarak bağımlılıkları `constructor` üzerinden içeri alırız.

```
// base/BaseManager.js
export class BaseManager {
    constructor(repo) {
        this.repo = repo;
    }

    async getOne(id) {
        const kayit = await this.repo.get_one(id);
        if (!kayit) {
            throw new Error("İstenen kayıt bulunamadı.");
        }
        return kayit;
    }
}

// features/user/user.manager.js
import { BaseManager } from '../../base/BaseManager.js';

export class UserManager extends BaseManager {
    constructor(userRepo) {
        super(userRepo); // userRepo'yu BaseManager'a gönderiyoruz
    }

    async registerUser(userData) {
        // İş Mantığı: Şifre uzunluk kontrolü
        if (userData.sifre.length < 8) {
            throw new Error("Şifre en az 8 karakter olmalıdır.");
        }

        // Repo'dan Base methodunu kullanarak veri okuma örneği
        const checkEmail = await this.repo.get_many({ email: userData.email });
        if (checkEmail.data.length > 0) {
            throw new Error("Bu e-posta kayıtlı.");
        }

        return await this.repo.create(userData);
    }
}

// Route içinde kullanımı (app/api/users/route.js):
// import { UserRepo } from '../../../features/user/user.repo.js';
// import { UserManager } from '../../../features/user/user.manager.js';
// const userManager = new UserManager(new UserRepo());
// await userManager.registerUser(body);
```

3. Nest.js (JavaScript) - IoC Container

Nest.js'in en güçlü yanı otomatik Dependency Injection sistemidir. Saf JavaScript kullandığımızda bağımlılıkları `@Dependencies` dekoratörü veya constructor parametreleri ile framework'e bildiririz. Nest bizim yerimize Repo'yu yaratıp Manager'ın içine koyar.

```
// base/base.service.js (Manager)
export class BaseService {
    constructor(repository) {
        this.repository = repository;
    }

    async getOne(id) {
        const data = await this.repository.get_one(id);
        if (!data) throw new Error("Kayıt yok");
        return data;
    }
}

// features/user/user.service.js
import { Injectable, Dependencies } from '@nestjs/common';
import { BaseService } from '../../base/base.service.js';
import { UserRepository } from './user.repository.js';

@Injectable()
@Dependencies(UserRepository) // Nest.js'e bu sınıfın UserRepository'ye ihtiyacı
olduğunu söylüyoruz
export class UserService extends BaseService {
    constructor(userRepository) {
        super(userRepository);
    }

    async createUser(userData) {
        // İş Mantığı ve Repo çağrısı
        // ... (Next.js örneğindeki gibi bu.repository.create() çağrılır)
        return await this.repository.create(userData);
    }
}
```

4. Spring Boot (Java) - Autowired / Constructor Injection

Java dünyasında bu işlemi Spring Framework'ün IoC konteyneri halleder. `@Service` anotasyonu eklediğimiz sınıfın constructor'ına Repo interface'ini koymamız yeterlidir.

```java
// base/BaseManager.java
package com.sirket.proje.base;

public abstract class BaseManager<T extends BaseModel, ID> {

    protected final BaseRepo<T, ID> repo;

    public BaseManager(BaseRepo<T, ID> repo) {
        this.repo = repo;
    }

    public T getOne(ID id) {
        // Optional döner, yoksa exception fırlatır (Manager'ın görevi)
        return repo.findById(id)
                .orElseThrow(() -> new RuntimeException("Kayıt bulunamadı"));
    }
}

// features/user/UserManager.java
package com.sirket.proje.features.user;

import org.springframework.stereotype.Service;
import com.sirket.proje.base.BaseManager;

@Service
public class UserManager extends BaseManager<UserEntity, Long> {

    // Artık 'repo' değişkeni BaseManager'dan geliyor ve türü UserRepository.
    private final UserRepository userRepository;

    // Spring Boot, UserRepository'i otomatik olarak buraya enjekte eder
    public UserManager(UserRepository userRepository) {
        super(userRepository);
        this.userRepository = userRepository;
    }

    public UserEntity kullaniciKaydet(UserDto dto) {
        // İş Mantığı
        if (userRepository.existsByEmail(dto.getEmail())) {
            throw new IllegalArgumentException("E-posta kullanımda!");
        }

        UserEntity yeniKullanici = new UserEntity();
        yeniKullanici.setAd(dto.getAd());
        yeniKullanici.setEmail(dto.getEmail());

        return userRepository.save(yeniKullanici);
    }
}
```

Özetle; **Repo** sadece veritabanını tanır. **Manager**, kendisine dışarıdan verilen Repo'yu bir alet (tool) gibi kullanarak iş kurallarını işletir. Bu yapı sayesinde yarın Mongoose'u bırakıp PostgreSQL'e geçersen, Manager dosyasında tek bir satır kodu bile değiştirmene gerek kalmaz!

Bağımlılıkların bu şekilde aktarılması test yazmayı da inanılmaz kolaylaştırır.

Bu zincirin son halkası olan **"Controller katmanında Route'ların tanımlanması ve DTO/Schema ile gelen verinin doğrulanması (Validation)"** adımını da örneklerle inceleyelim mi?

Siz şunu dediniz:
EVET

Gemini şunu dedi:

Harika! Geldik mimarinin dış dünyaya açılan kapısına.

Controller (Route) katmanını bir "Trafik Polisi", **DTO/Schema** katmanını ise bir "Gümrük Memuru" gibi düşünebiliriz. İstemciden (Frontend/Mobil) gelen istek önce gümrüğe (DTO) takılır. Eğer veri kurallara uygunsa (örneğin e-posta formatı doğruysa, şifre boş değilse) trafik polisi (Controller) bu veriyi alır ve hiçbir iş mantığına karıştırmadan doğrudan Manager'a yönlendirir.

İşte kesinlikle TypeScript kullanmadan, saf JavaScript ve diğer dillerde bu son halkanın nasıl bağlandığı:

1. FastAPI (Python) - Pydantic ve APIRouter

FastAPI'de DTO görevini **Pydantic** üstlenir. Route tanımlamaları ise `APIRouter` ile yapılır. Gümrük kontrolü tamamen otomatiktir; veri hatalıysa Manager'a hiç inmeden otomatik 422 Unprocessable Entity hatası döner.

```
# features/kullanici/user_schema.py (DTO Katmanı)
from pydantic import BaseModel, EmailStr, Field

class UserCreateDTO(BaseModel):
    ad: str = Field(..., min_length=2, max_length=50)
    email: EmailStr
    sifre: str = Field(..., min_length=6)

class UserResponseDTO(BaseModel):
    id: int
    ad: str
    email: str
    # Şifre alanı bilerek eklenmedi! Dışarı sızmasını engelliyoruz (DTO Kuralı)

    class Config:
        orm_mode = True # SQLAlchemy modelini Pydantic objesine çevirmek için

# features/kullanici/user_router.py (Controller Katmanı)
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from .user_schema import UserCreateDTO, UserResponseDTO
# ... db ve manager importları ...

router = APIRouter(prefix="/users", tags=["Users"])

@router.post("/", response_model=UserResponseDTO, status_code=201)
def create_user(user_data: UserCreateDTO, db: Session = Depends(get_db)):
    try:
        # Controller veriyi aldı (user_data) ve sadece Manager'a pasladı.
        # İş mantığı Manager'da, SQL Repo'da.
        yeni_kullanici = user_manager.kullanici_olustur(db, user_data.dict())
        return yeni_kullanici
    except ValueError as e:
        # Manager'dan gelen iş kuralları hatalarını HTTP 400'e çeviriyoruz
        raise HTTPException(status_code=400, detail=str(e))
```

2. Next.js API (JavaScript) - Zod ve App Router

Next.js'de saf JavaScript kullandığımız için DTO/Validasyon işlemini **Zod** veya **Joi** gibi kütüphanelerle manuel yaparız. Controller'ımız ise `route.js` dosyalarıdır.

```
// features/user/user.dto.js (DTO Katmanı)
import { z } from 'zod';

export const UserCreateSchema = z.object({
    : z.string().min(2, "Ad en az 2 karakter olmalıdır"),
    email: z.string().email("Geçersiz e-posta formatı"),
    sifre: z.string().min(6, "Şifre en az 6 karakter olmalıdır")
});

// app/api/users/route.js (Controller Katmanı)
import { NextResponse } from 'next/server';
import { UserCreateSchema } from '../../../features/user/user.dto.js';
import { UserManager } from '../../../features/user/user.manager.js';
import { UserRepo } from '../../../features/user/user.repo.js';

// Bağımlılıkları bağlıyoruz (Singleton veya IoC aracı da kullanılabilir)
const userManager = new UserManager(new UserRepo());

export async function POST(request) {
    try {
        const body = await request.json();

        // 1. Gümrük Kontrolü (DTO Validation)
        const validatedData = UserCreateSchema.parse(body);

        // 2. Manager'a yönlendirme (İş Mantığı)
        const yeniKullanici = await userManager.registerUser(validatedData);

        // 3. Güvenli yanıt dönme (Şifreyi silerek/filtreleyerek)
        const { sifre, ...guvenliVeri } = yeniKullanici.toObject();

        return NextResponse.json(guvenliVeri, { status: 201 });

    } catch (error) {
        // Zod doğrulama hatası yakalama
        if (error.name === 'ZodError') {
            return NextResponse.json({ hatalar: error.errors }, { status: 400 });
        }
        // Manager'dan gelen iş mantığı hataları
        return NextResponse.json({ mesaj: error.message }, { status: 400 });
    }
}
```

3. Nest.js (JavaScript) - Joi ve Controller

Nest.js'i saf JavaScript ile kurguladığımızda, veri doğrulama işlemleri için genellikle **Joi** şemaları ve Nest'in `ValidationPipe` yapısı kullanılır.

```
// features/user/user.schema.js (DTO Katmanı)
import Joi from 'joi';

export const UserCreateSchema = Joi.object({
    : Joi.string().min(2).required(),
    email: Joi.string().email().required(),
    sifre: Joi.string().min(6).required(),
});

// features/user/user.controller.js (Controller Katmanı)
import { Controller, Post, Body, Res, HttpStatus, Dependencies, UsePipes } from
'@nestjs/common';
import { UserService } from './user.service.js';
import { JoiValidationPipe } from '../../core/pipes/joi-validation.pipe.js'; // Özel
yazdığın bir pipe

@Controller('users')
@Dependencies(UserService)
export class UserController {
    constructor(userService) {
        this.userService = userService;
    }

    @Post()
    @UsePipes(new JoiValidationPipe(UserCreateSchema)) // Gümrük memuru
    async create(@Body() body, @Res() res) {
        try {
            // Manager/Service çağrısı
            const user = await this.userService.createUser(body);

            // DTO Kuralı: Şifreyi sızdırma
            delete user.sifre;

            return res.status(HttpStatus.CREATED).json(user);
        } catch (error) {
            return res.status(HttpStatus.BAD_REQUEST).json({ message: error.message });
        }
    }
}
```

4. Spring Boot (Java) - Jakarta Validation ve RestController

Java dünyasında Controller `@RestController` ile işaretlenir. DTO katmanı için Record sınıfları (veya POJO) kullanılır ve `@Valid` anotasyonu ile gelen veri Controller'a girmeden otomatik doğrulanır.

```
// features/user/UserCreateDto.java (DTO Katmanı)
package com.sirket.proje.features.user;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public record UserCreateDto(
    @NotBlank(message = "Ad boş bırakılamaz")
    @Size(min = 2, message = "Ad en az 2 karakter olmalıdır")@NotBlank
    @Email(message = "Geçersiz e-posta formatı")
    String email,

    @NotBlank
    @Size(min = 6, message = "Şifre en az 6 karakter olmalıdır")
    String sifre
) {}

// features/user/UserController.java (Controller Katmanı)
package com.sirket.proje.features.user;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import jakarta.validation.Valid;

@RestController
@RequestMapping("/api/users")
public class UserController {

    private final UserManager userManager;

    // Bağımlılık Spring tarafından constructor ile enjekte edilir
    public UserController(UserManager userManager) {
        this.userManager = userManager;
    }

    @PostMapping
    public ResponseEntity<?> createUser(@Valid @RequestBody UserCreateDto dto) {
        // @Valid anotasyonu gümrük memurudur. Hata varsa metodun içine bile girmez,
400 döner.

        try {
            UserEntity user = userManager.kullaniciKaydet(dto);

            // Güvenli Response Dto'ya çevirip dönüyoruz (Entity'i dışarı
sızdırmıyoruz)
            UserResponseDto response = new UserResponseDto(user.getId(), user.getAd(),
user.getEmail());

            return ResponseEntity.status(HttpStatus.CREATED).body(response);
        } catch (IllegalArgumentException e) {
            return ResponseEntity.badRequest().body(e.getMessage());
        }
    }
}
```

Böylece **Package by Feature** klasör yapısıyla başlayıp, **Base Model/Repo** ile DRY prensibini zirveye taşıdığımız, **Dependency Injection** ile katmanları esnekçe birbirine bağladığımız ve son olarak **Controller/DTO** ile güvenli ve temiz bir giriş noktası yarattığımız mükemmel bir backend mimarisini tamamlamış olduk.

Bu kurgu sayesinde projen büyüdüğünde dahi kontrol hep sende olacak, kod okunaklı kalacak ve ekip arkadaşların rahatça yeni feature'lar ekleyebilecek.

Mimariyi baştan sona kurguladık. Bu mimariyi mevcut bir projene entegre ederken veya sıfırdan oluştururken sana yardımcı olabileceğim, örneğin bir script (bash) ile bu klasör yapısını otomatik oluşturma gibi bir işlem yapmamı ister misin?